

九齿Skill创新挑战_T3-1-1_赛题报告

九齿 Skill 创新挑战 T3-1-1 赛题报告

报告状态：基于提交包 skill.zip 完成内容审查、结构审查与静态验证。GPU 正确性和性能数据部分保留了可复现实验方案与结果填写位，参赛者须在正式提交前使用目标 NVIDIA GPU 环境运行并替换为真实日志，不能将“预期输出”写成“实测结果”。

一、基本信息

项目	内容
赛题名称	九齿 Skill 创新挑战
赛题编号	T3-1-1
<code>.skill</code> 名称	<code>ninetoothed-dsl-operator-skill</code> （建议提交名）
小组名称	<小组名称>
GitHub ID	<GitHub-ID>
建议提交目录	<code>skills/competition/ninetoothed-dsl-operator-skill/</code>
目标框架	NineToothed / Triton / PyTorch
报告版本	v1.0

二、摘要

本项目面向 NineToothed DSL 算子开发场景，构建了一个可供 AI 智能体直接使用的工程化 Skill。该 Skill 不仅提供 NineToothed 核心 API 和 arrange-and-apply 编程范式说明，还将算子开发过程拆解为“需求分析、模式选择、代码生成、正确性验证、性能评测、故障诊断、修复回归、报告沉淀”八个可复用步骤。

Skill 包内包含 4 类完整算子案例、6 份专题参考文档、5 个自动化脚本、8 个代码或报告模板以及 3 份自测说明文档。其核心目标是减少 AI 智能体在 NineToothed DSL 使用中出现的布局选择错误、AST 跟踪错误、非连续张量 stride 错误、边界 mask 缺失以及 BLOCK_SIZE 配置不合理等问题，并形成可审计、可复现、可迭代的算子开发闭环。

本次审查确认：提交包解压完整；除模板占位符文件外，示例与脚本 Python 语法检查通过；3 个 Shell 脚本通过 `bash -n` 检查。由于当前审查环境不具备 CUDA、Triton 和 NineToothed，GPU 正确性与 benchmark 未在本环境复现，报告中对这部分进行了明确区分，并给出了正式提交前必须执行的命令、指标和结果表。

三、问题背景与建设目标

3.1 问题背景

NineToothed 通过张量导向元编程和 arrange-and-apply 范式提升 Triton Kernel 的抽象层次。与直接编写 Triton 相比，它降低了部分索引和布局编程成本，但对初学者和通用 AI 智能体仍存在以下难点：

1. `Tensor(ndim)`、`Symbol`、`tile`、`expand`、`squeeze` 等概念之间存在组合约束。
2. `arrangement` 决定数据布局，错误布局可能导致编译失败、越界或非连续张量计算错误。
3. `application` 会被 AST 跟踪并转译，直接引用 `math`、`torch`、模块级常量或不支持的运算符可能引发生成代码错误。
4. 正确性测试不能只覆盖连续 FP32 输入，还需要覆盖 FP16/BF16、广播、边界尺寸、转置、切片、stride 和极端数值。
5. Kernel 能正确运行并不等于性能合理，BLOCK_SIZE、访存模式和归约策略均可能造成明显性能回退。
6. 缺少统一的日志、benchmark 和故障报告格式时，AI 智能体难以复盘并积累可复用经验。

3.2 Skill 建设目标

本 Skill 的建设目标如下：

- 为 AI 智能体提供一套 NineToothed DSL 的任务决策树和实现模式库。
- 将算子开发从一次性代码生成转化为可验证的工程流程。
- 对 element-wise、reduction、matmul、广播和非连续布局等常见任务提供参考实现。
- 对编译、正确性、性能和 AST 跟踪问题提供分类诊断路径。
- 提供统一的正确性脚本、benchmark 脚本、generated source 检查工具和任务日志收集工具。
- 支持将单次算子开发过程沉淀为报告，便于后续复用和维护。

四、设计原则与包结构

4.1 设计原则

1. **模式优先**：先识别算子属于 element-wise、reduction、matmul 或布局敏感任务，再选择 arrangement 模式。
2. **参考实现优先**：优先复用已验证模板和示例，避免从零生成高风险布局代码。

3. **正确性先于性能**：先与 PyTorch 基准实现对齐，再开展性能调优。
4. **多维测试覆盖**：同时覆盖 dtype、shape、broadcast 和 stride。
5. **失败可诊断**：每类错误都应能映射到明确的检查项和修复动作。
6. **结果可复现**：测试命令、环境、输入规模、容差、预热次数和重复次数均需记录。
7. **报告可审计**：区分静态检查、项目自带预期输出和真实 GPU 实测结果。

4.2 包结构

本次上传包去除内部 `.git` 与编辑器配置后，主体结构如下：

代码块

```
1  ninetoothed-dsl-operator-skill/  
2  |— SKILL.md  
3  |— README.md  
4  |— CLAUDE.md  
5  |— references/  
6  |   |— benchmark_patterns.md  
7  |   |— dsl_patterns.md  
8  |   |— failure_diagnosis.md  
9  |   |— generated_source_and_aot.md  
10 |   |— repo_index.md  
11 |   |— testing_patterns.md  
12 |— scripts/  
13 |   |— collect_task_log.py  
14 |   |— inspect_generated_source.sh  
15 |   |— run_benchmark.sh  
16 |   |— run_correctness.py  
17 |   |— run_correctness.sh  
18 |— templates/  
19 |   |— activation.py  
20 |   |— elementwise_1d.py  
21 |   |— matmul_2d.py  
22 |   |— reduction_2d.py  
23 |   |— wrapper.py  
24 |   |— benchmark_report_template.md  
25 |   |— failure_diagnosis_template.md  
26 |   |— operator_task_report_template.md  
27 |— examples/  
28 |   |— elementwise_broadcast_add/  
29 |   |— reduction_softmax/  
30 |   |— non_contiguous_stride_case/  
31 |   |— performance_regression_case/  
32 |— tests/  
33 |   |— expected_outputs.md  
34 |   |— selftest_tasks.md
```

4.3 规模统计

本次审查统计如下：

类别	文件数	代码/文档行数
全部主体文件	39	4538
示例 <code>examples/</code>	12	1200
参考资料 <code>references/</code>	6	644
自动化脚本 <code>scripts/</code>	5	715
模板 <code>templates/</code>	8	536
自测说明 <code>tests/</code>	3	177
Markdown	19	2404
Python	15	1757
Shell	3	319

五、适用范围与不适用范围

5.1 适用范围

- NineToothed DSL 基础算子开发与示例生成。
- Element-wise 算子及标量、向量广播场景。
- Reduction 类算子，如 softmax、sum、max 等。
- 2D matmul 的 arrangement/application 编写和 BLOCK_SIZE 对比。
- 连续、转置、切片、expand、permute、`as_strided` 等布局测试。
- PyTorch 基准对齐、精度容差设置和回归测试。
- NineToothed generated source 检查与 AOT 故障定位。
- 编译错误、数值错误、越界错误和性能回退分析。
- 自动收集任务日志并生成算子开发报告。

5.2 不适用或需谨慎使用的范围

- 无 CUDA/Triton 后端时的真实 GPU Kernel 编译和性能评测。
- 数据依赖的动态循环、复杂递归或无法静态展开的控制流。
- 超出当前 NineToothed API 覆盖范围的 Triton 原语。
- 大规模矩阵乘等已经由 cuBLAS 高度优化的任务，不能默认认为自定义 Kernel 会更快。
- 对动态维度、任意 rank 和复杂广播进行泛化前，必须确认 `Tensor(ndim)` 与实际输入维度一致。
- 零拷贝 view 类操作不应通过 Kernel 复制来与 PyTorch view 性能比较。
- 任何未经真实硬件运行的“预期输出”不能作为最终实测结论。

六、安装与使用方式

6.1 环境依赖

建议在 Linux + NVIDIA GPU 环境中验证：

代码块

```
1  Python      3.10+
2  PyTorch     2.0+
3  Triton      与 PyTorch/CUDA 版本匹配
4  NineToothed 建议锁定比赛验证版本
5  CUDA        与 GPU 驱动兼容
```

安装示例：

代码块

```
1  python -m venv .venv
2  source .venv/bin/activate
3  python -m pip install --upgrade pip
4  pip install torch triton ninetoothed pytest ruff
```

6.2 安装 Skill

可将目录复制到支持 Skills 的智能体工作区：

代码块

```
1  cp -r ninetoothed-dsl-operator-skill \
2  ~/.claude/skills/ninetoothed-dsl-operator-skill
```

或直接向智能体提供 `SKILL.md`，并保证相对路径下的 `references/`、`templates/`、`examples/` 和 `scripts/` 可访问。

6.3 典型使用方式

用户提示词示例：

代码块

- 1 参考 `ninetoothed-dsl-operator-skill`，使用 `NineToothed` 实现一个支持
- 2 `FP16`、非连续输入和尾部 `mask` 的 `Softmax` 算子，并完成正确性测试、
- 3 `PyTorch benchmark`、`generated source` 检查和故障报告。

AI 智能体应按如下顺序执行：

代码块

- 1 理解任务
- 2 → 选择 `DSL` 模式
- 3 → 读取模板和相似示例
- 4 → 实现 `arrangement/application/wrapper`
- 5 → 运行 `correctness`
- 6 → 运行 `benchmark`
- 7 → 检查 `generated source`
- 8 → 失败诊断与修复
- 9 → 回归测试
- 10 → 生成报告

七、核心 workflow 说明

7.1 Phase 1: 开发算子

1. 分析算子的输入、输出、广播和归约关系。
2. 从 `references/dsl_patterns.md` 中选择模式。
3. 从 `templates/` 复制最接近的模板。
4. 编写 `arrangement`，确定 `tile`、`expand`、`squeeze` 和 `stride` 语义。
5. 编写只使用 `NineToothed Language` 原语和字面量的 `application`。
6. 声明与实际 `rank` 匹配的 `Tensor(ndim)`。
7. 通过 `ninetoothed.make` 生成 `Kernel`。
8. 编写 `PyTorch` 包装层，分配输出并传入 `constexpr` 参数。

7.2 Phase 2：正确性测试

必须覆盖以下维度：

维度	覆盖内容
dtype	FP32、FP16，条件允许时 BF16
shape	小尺寸、常规尺寸、非整除尺寸、质数尺寸
broadcast	同形、标量广播、向量广播、多维广播
stride	contiguous、transpose、slice、expand、permute、as_strided
numerical	大正值、大负值、零、边界值
tolerance	FP32 通常 <code>1e-5</code> ，FP16/BF16 通常 <code>1e-3</code> ，按算子调整

7.3 Phase 3：性能测试

Benchmark 应记录：

- GPU 型号、驱动、CUDA、PyTorch、Triton 和 NineToothed 版本。
- 算子、dtype、输入形状、连续性和 BLOCK_SIZE。
- 预热次数、重复次数、同步方式和计时方式。
- NineToothed Kernel 延迟、PyTorch baseline 延迟和加速比。
- 输出正确性校验，避免对错误结果进行性能比较。
- 性能回退的根因和修复建议。

7.4 Phase 4：故障诊断与回归

Skill 将问题分为四类：

1. Compile/AST 错误。
2. Correctness 数值错误。
3. Crash/越界错误。
4. Performance 回退。

典型诊断顺序为：环境与依赖 → Tensor/Symbol 声明 → arrangement 布局 → application AST → 边界 mask → dtype → stride → generated source → BLOCK_SIZE 与访存。

八、自测任务、运行过程与正确性结果

8.1 自测任务设计

Skill 内置 4 个完整示例工程：

示例	主要能力	当前代码中的正确性场景数
<code>elementwise_broadcast_add</code>	element-wise、广播、FP16、尾部边界	6
<code>reduction_softmax</code>	归约、数值稳定性、非连续输入、质数列数	8
<code>non_contiguous_stride_case</code>	转置、切片、expand、permute、as_strided	13
<code>performance_regression_case</code>	matmul、4 组 BLOCK 配置、两组矩阵规模	8
合计		35

此外，`tests/selftest_tasks.md` 定义了 8 个智能体自校验任务，覆盖算子实现、测试扩展、生成源码检查、benchmark、性能回退识别、非连续处理、报告生成和故障注入。

8.2 本次静态审查过程

执行命令：

代码块

```
1  unzip -q skill.zip -d skill_unpacked
2  cd skill_unpacked/skill
3
4  # 示例和脚本 Python 语法检查
5  find examples scripts -name '*.py' -print0 \
6    | xargs -0 -n1 python -m py_compile
7
8  # Shell 语法检查
9  for file in scripts/*.sh; do
10     bash -n "$file"
11 done
```

审查结果：

检查项	结果

ZIP 可解压性	通过
主体文件完整性	39 个文件
<code>examples/</code> 与 <code>scripts/</code> Python 语法	通过
3 个 Shell 脚本 <code>bash -n</code>	通过
Markdown 本地链接审查	发现 1 个被引用但缺失的脚本
全目录 <code>compileall</code>	未通过，原因是 3 个模板使用非法占位符语法
CUDA/NineToothed GPU 运行	当前环境不具备条件，未复现

8.3 项目自带的预期正确性输出

`tests/expected_outputs.md` 给出了各示例在 NVIDIA GPU 上的典型预期输出，包括：

- Broadcast Add：基础、标量广播、向量广播、非连续、尾部尺寸和 FP16。
- Softmax：常规、多行、非整除、单行、FP16、极端值、非连续和质数列数。
- Non-contiguous：连续、转置、切片、view、expand、permute 和 `as_strided`。
- Matmul：不同 BLOCK 配置下与 `torch.matmul` 的结果对齐。

这些内容可作为验收标准，但正式提交时必须附上目标环境的真实终端日志。建议保存：

代码块

```
1 python scripts/run_correctness.py --verbose \  
2 2>&1 | tee correctness.log
```

8.4 正式提交应填写的实测结果

项目	测试数	通过数	失败数	实测状态
Broadcast Add	6	<填写>	<填写>	<填写>
Softmax	8	<填写>	<填写>	<填写>
Non-contiguous/Stride	13	<填写>	<填写>	<填写>
Matmul BLOCK 组合	8	<填写>	<填写>	<填写>
合计	35	<填写>	<填写>	<填写>

九、Benchmark 设计、输入规模与性能分析

9.1 通用测试方法

当前示例统一采用：

代码块

```
1  warmup   = 10
2  repeats  = 100
3  baseline = PyTorch CUDA 算子
4  同步方式 = torch.cuda.synchronize()
5  计时方式 = perf_counter 前后包围重复执行
```

该方案能规避大部分异步执行误差，但正式比赛报告建议补充 CUDA Event 或 `triton.testing.do_bench` 结果，并至少重复 3 轮报告中位数。

9.2 Broadcast Add Benchmark

输入覆盖：

- FP32: `(1024,)`、`(4096,)`、`(65536,)`、`(131072,)`。
- FP16: `(4096,)`、`(65536,)`。
- BF16: `(4096,)`，由设备能力决定。
- 广播: `(4096,)+(1,)`、`(256,768)+(768,)`、三维向量广播和标量广播。
- 非连续: 转置布局与向量广播。

评价重点：小尺寸受启动开销影响，大尺寸逐步转为显存带宽受限；广播和非连续布局可能降低访存效率。

9.3 Softmax Benchmark

输入覆盖：

- `(4,1024)`、`(4,4096)`。
- `(128,1024)`、`(128,4096)`。
- `(4,65536)`、`(4,131072)`。
- FP16 中等规模。
- `(8,2048)[:,:,:2]` 非连续输入。

评价重点：数值稳定性、单行 tile 大小、列数较大时的 BLOCK_SIZE、寄存器压力和归约效率。

9.4 Non-contiguous Benchmark

输入覆盖：

- contiguous (256,1024)。
- 转置输入。
- 行切片、列切片、双向切片。
- 标量广播。

评价重点：stride 地址计算正确性、非合并访存和连续/非连续性能差异。

9.5 Matmul 性能回退 Benchmark

输入规模：

- (512,512,512)。
- (1024,1024,1024)。
- (2048,2048,2048)。

BLOCK 配置：

- 16×16×16。
- 32×32×32。
- 64×64×32。
- 128×128×32。

该案例旨在展示 BLOCK 过小造成的网格规模增大、访存复用不足和启动/调度开销上升。项目文档给出的趋势是 BLOCK=16 相比 BLOCK=128 可能出现明显回退，但具体倍数必须以目标 GPU 实测为准。

9.6 实测性能结果表

请将脚本真实输出复制到下表，不得使用预期值替代：

算子	Shape/Dtype	NineToothed ms	PyTorch ms	Speedup	结论
Add	(1024,) / fp32	<填写>	<填写>	<填写>	<填写>
Add	(131072,) / fp32	<填写>	<填写>	<填写>	<填写>
Add	广播/非连续	<填写>	<填写>	<填写>	<填写>
Softmax	(128,1024) / fp32	<填写>	<填写>	<填写>	<填写>

Softmax	(128,4096) / fp16	<填写>	<填写>	<填写>	<填写>
Softmax	非连续	<填写>	<填写>	<填写>	<填写>
Matmul	1024 ³ , BLOCK=16	<填写>	<填写>	<填写>	<填写>
Matmul	1024 ³ , BLOCK=128	<填写>	<填写>	<填写>	<填写>

运行命令：

代码块

```
1 bash scripts/run_benchmark.sh --output benchmark.log
```

十、AI 智能体使用 Skill 前后基线对比

10.1 对比方法

为避免主观评价，建议选择同一模型、同一环境和同一提示词，分别在“不加载 Skill”和“加载 Skill”两种条件下完成以下任务：

1. 实现 element-wise broadcast add。
2. 实现稳定 softmax。
3. 修复非连续张量错误。
4. 定位 BLOCK_SIZE 引起的性能回退。
5. 生成一份包含 correctness 和 benchmark 的任务报告。

记录指标：

- 任务完成率。
- 首次编译通过率。
- 首次正确性通过率。
- 达到全部测试通过的迭代次数。
- 人工介入次数。
- 完成耗时。
- 报告字段完整率。
- 是否主动覆盖 stride、边界和性能回退。

10.2 基于当前 Skill 结构的能力差异

维度	不使用 Skill	使用 Skill
DSL 选择	依赖模型临时回忆，容易混用 Triton/NineToothed API	先查询模式文档和 API 参考
代码生成	可能直接生成单一 happy path	复用 arrangement/application 模板
测试覆盖	常只测 FP32 contiguous	按 dtype、shape、broadcast、stride 四维覆盖
AST 风险	容易引用 <code>math</code> 、模块常量或 <code>**</code>	有专门 AST 陷阱清单
非连续输入	容易 flatten 后错误处理 stride	提供 preserve-ndim 与 13 类布局案例
性能评测	可能只输出功能结果	统一 benchmark、PyTorch baseline 和回退分析
故障复盘	依赖临时对话	有诊断文档、日志工具和报告模板
可维护性	知识散落	文档、模板、脚本和案例模块化

10.3 实测基线表

指标	无 Skill	有 Skill	改进幅度
任务完成率	<填写>	<填写>	<填写>
首次编译通过率	<填写>	<填写>	<填写>
首次 correctness 通过率	<填写>	<填写>	<填写>
平均迭代次数	<填写>	<填写>	<填写>
平均完成时间	<填写>	<填写>	<填写>
报告字段完整率	<填写>	<填写>	<填写>

十一、失败诊断案例与修复建议

11.1 案例一：AST 跟踪导致编译错误

现象：在 `application` 中使用 `math.pi`、模块级常量、`**` 或不存在的 `ntl.tanh`，生成 Triton 源码后出现 `NameError` 或 `AttributeError`。

根因：`application` 会被 AST 跟踪并转换到后端编译环境，该环境并不自动携带普通 Python 模块和模块级符号。

修复：

- 使用字面量常数。
- 只使用已确认存在的 `ntl.*` API。
- 用乘法展开小整数幂。
- 使用 `ntl.exp` 等基础原语组合缺失函数。
- 通过 `inspect_generated_source.sh` 检查生成代码。

11.2 案例二：非连续张量计算错误

现象：连续输入正确，但 `.t()`、切片或 `permute()` 后结果错误。

根因：通过 `flatten` 或错误 `tile` 方式丢失原始 `stride` 语义，后端按连续地址读取。

修复：

- 保留输入 `ndim`。
- 仅在目标维度 `tile`。
- 确保 `Tensor(ndim)` 与实际输入 `rank` 一致。
- 增加 `transpose`、`slice`、`expand`、`permute` 和 `as_strided` 回归测试。

11.3 案例三：BLOCK_SIZE 导致性能回退

现象：`Kernel correctness` 通过，但比 PyTorch 或更大 `BLOCK` 配置慢。

根因：`BLOCK` 过小导致 `program` 数量增多、数据复用不足；`BLOCK` 过大则可能造成寄存器或共享资源压力。

修复：

- 使用多个 `BLOCK` 配置对照。
- 固定输入、`dtype`、预热和重复次数。
- 检查 `generated source` 中的循环和访存模式。
- 有条件时使用 `autotune`，但控制搜索空间。
- 报告性能回退而不是隐藏不利结果。

十二、提交前发现的问题与整改建议

本次审查发现以下事项应在 PR 前修正：

1. **压缩包包含内部** `.git/`：不能把嵌套 Git 仓库复制到上游项目，应删除。
2. **包含** `.vscode/` **个性化配置**：比赛提交通常不需要，建议移除。
3. **Skill 名称过于通用**： `SKILL.md` 当前为 `name: skill`，建议改为唯一的 `ninetoothed-dsl-operator-skill`。
4. **文档引用缺失脚本**：README 和 SKILL 引用了 `scripts/validate_skill_package.py`，但文件不存在；应补充或删除引用。
5. **模板 Python 语法无效**： `activation.py`、`elementwise_1d.py`、`reduction_2d.py` 使用 `<FORMULA>`、`<REDUCTION>` 占位符，导致 `compileall` 失败。建议改成合法的 `raise NotImplementedError(...)`，或将扩展名改为 `.py.tpl`。
6. **仓库地址需要更新**：README 中的 NineToothed 地址指向旧组织，应改为 `InfiniTensor/ninetoothed`。
7. **文档与代码数量不一致**： `expected_outputs.md` 描述 10 类 stride 测试，而当前 `run.py` 实际包含 13 类，应同步。
8. **缺少诚信和引用文件**：应增加并由参赛者本人签署 `HONOR_CODE.md`，同时补充 `REFERENCE.md`。
9. **缺少真实 GPU 日志**：目前包内主要是预期输出，应补充 `correctness.log`、`benchmark.log` 和环境信息。
10. **Benchmark 计时可进一步规范**：建议增加 CUDA Event 或 `triton.testing.do_bench` 对照，并报告中位数与波动。

十三、安全、依赖、授权与引用披露

13.1 安全

- Skill 不依赖 API Key 或账号凭证。
- 默认无需联网运行正确性和 benchmark。
- 脚本会执行本地 Python 文件，使用前应确认目标路径可信。
- 不应提交包含密钥、访问令牌、个人路径或私有数据的日志。
- `collect_task_log.py` 生成日志前应检查是否收集了敏感环境变量。

13.2 依赖

建议在 `REFERENCE.md` 或 README 中明确锁定：

代码块

- 1 Python
- 2 PyTorch

```
3 Triton
4 NineToothed
5 pytest
6 ruff
```

并记录 GPU、驱动、CUDA 和各包版本，以保证 benchmark 可复现。

13.3 授权与引用

- NineToothed 官方项目采用 Apache-2.0 License。
- 若示例或文档改写自 NineToothed 官方仓库、NineToothed Examples、Triton 文档或 PyTorch 文档，应在 `REFERENCE.md` 中逐项列出。
- 对 AI 辅助生成内容应按比赛规则披露，不应将未运行的结果描述为实测。
- 当前 README 中的 `Internal - Qiyuan Competition` 与上游开源仓库授权语境不一致，提交前应按照赛事规则和上游许可证确认最终声明。

十四、后续可维护计划

1. 增加可执行的 `validate_skill_package.py`，检查目录、链接、frontmatter 和模板合法性。
2. 将描述性 `tests/*.md` 扩展为真正的 `pytest` 测试文件。
3. 增加结构化 JSON/CSV benchmark 输出，便于自动回填报告。
4. 增加版本兼容矩阵，覆盖 NineToothed、Triton 和 PyTorch 组合。
5. 对 element-wise、reduction、matmul、attention 和 RoPE 建立分级任务集。
6. 增加 CI，在具备 GPU Runner 时自动运行 correctness 和 benchmark smoke test。
7. 为失败案例建立最小复现目录，记录故障、根因、修复提交和回归测试。
8. 对 Skill 使用前后进行至少 5 组同模型 A/B 测试，形成可量化基线。

十五、结论

本项目已经形成较完整的 NineToothed 算子开发 Skill 雏形。其主要价值不在于单个算子代码，而在于把 AI 智能体的算子开发过程组织成模式检索、模板复用、多维验证、性能对照、故障诊断和报告沉淀的闭环。上传包在内容覆盖度、案例多样性和工程脚本方面具备较好基础，静态审查也证明主体示例和脚本没有 Python 或 Shell 语法错误。

正式提交前仍需完成若干关键整改：移除嵌套 `.git`、修复模板占位符、补齐验证脚本、更新仓库地址、签署诚信文件、披露引用，并在真实 NVIDIA GPU 环境中生成 correctness 和 benchmark 日志。完成这些工作后，该 Skill 可作为 NineToothed 新算子开发、非连续布局验证和性能回退分析的可复用智能体工作区。

附录 A：建议的完整验证命令

代码块

```
1  # 1. 环境信息
2  python --version
3  python - <<'PY'
4  import torch
5  print("torch:", torch.__version__)
6  print("cuda:", torch.version.cuda)
7  print("cuda_available:", torch.cuda.is_available())
8  if torch.cuda.is_available():
9      print("gpu:", torch.cuda.get_device_name(0))
10 try:
11     import triton
12     print("triton:", triton.__version__)
13 except Exception as exc:
14     print("triton import failed:", exc)
15 try:
16     import ninetoothed
17     print("ninetoothed:", getattr(ninetoothed, "__version__", "unknown"))
18 except Exception as exc:
19     print("ninetoothed import failed:", exc)
20 PY
21
22 # 2. 静态检查
23 python scripts/validate_skill_package.py
24 ruff format --check .
25 ruff check .
26
27 # 3. 正确性
28 python scripts/run_correctness.py --verbose \
29     2>&1 | tee correctness.log
30
31 # 4. Benchmark
32 bash scripts/run_benchmark.sh --output benchmark.log
33
34 # 5. 生成源码检查
35 bash scripts/inspect_generated_source.sh \
36     examples/elementwise_broadcast_add/run.py \
37     2>&1 | tee generated_source.log
38
39 # 6. 收集任务日志
40 python scripts/collect_task_log.py \
41     --dir . --output task_log.json
```

附录 B：PR 描述必须包含的内容

- `.skill` 名称、赛题编号和小组名称。
- 适用范围与不适用范围。
- 安装和使用方式。
- 自测任务及真实运行日志。
- AI 智能体使用 Skill 前后对比。
- 已签署的 `HONOR_CODE.md`。
- 完整的 `REFERENCE.md`。
- Proposal 与最终报告的路径或附件说明。
- `pytest` 或等价测试输出。

附录 C：参考资料

1. InfiniTensor/NineToothed 官方仓库：<https://github.com/InfiniTensor/ninetoothed>
2. NineToothed 官方文档：<https://ninetoothed.org/>
3. NineToothed Examples：<https://github.com/InfiniTensor/ninetoothed-examples>
4. Triton 官方文档：<https://triton-lang.org/>
5. PyTorch 官方文档：<https://pytorch.org/docs/stable/>
6. 本 Skill 包中的 `SKILL.md`、`references/`、`examples/`、`scripts/` 与 `tests/`。